VSEC ASSIGNMENTS

Experiment 1: Git and GitHub Operations Using Git Bash

Aim: To perform Git operations using Git Bash, initialize a repository, push code to GitHub, create a new branch, and demonstrate differences between branches.

a) Initialize a Git repository for a Data Science project

Step 1: Open Git Bash and create a project folder.

```
mkdir DataScienceProject
cd DataScienceProject
```

Step 2: Initialize Git repository.

```
git init
```

This initializes an empty Git repository in the project folder.

Step 3: Configure Git username and email.

```
git config --global user.name "Your Name"
git config --global user.email “your-email@gmail.com"
```

b) Push the repository to GitHub

1. Open GitHub.
2. Click on New Repository.
3. Enter repository name: `DataScienceProject`.
4. Select Public or Private.
5. Click Create Repository.

Connect your local repository to GitHub:

```
git remote add origin https://github.com/username/
DataScienceProject.git
```

Replace `username` with your GitHub username.

c) Create program1.py to print the multiplication table

Create a Python file:

```
touch program1.py
```

Open the file in VS Code or any text editor and write:

program1.py

```
num = int(input("Enter a number: “))

for i in range(1, 11):
     print(num, "x", i, "=", num * i)
```

Example output:

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
...
5 x 10 = 50
```

d) Commit the code and push changes to GitHub

Step 1: Check the status.

```
git status
```

Step 2: Add the file to the staging area.

```
git add program1.py
```

Step 3: Commit the changes.

```
git commit -m "Added multiplication table program”
```

Step 4: Rename the default branch to master.

```
git branch -M master
```

Step 5: Push changes to GitHub.

```
git push -u origin master
```

Step 6: Verify changes.

Open your GitHub repository and check whether `program1.py` is uploaded successfully.

e) Create a new branch GIT-EXAM and create program2

Step 1: Create and switch to a new branch.

```
git checkout -b GIT-EXAM
```

This creates a new branch named `GIT-EXAM` and switches to it.

Step 2: Create a new Python file.

```
touch program2.py
```

Step 3: Write a Python program to find the greatest of two numbers.

program2.py

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
if num1 > num2:
     print(num1, "is greatest")
elif num2 > num1:
     print(num2, "is greatest")
else:
     print("Both numbers are equal")
```

Example output:

```
Enter first number: 20
Enter second number: 10
20 is greatest
```

f) Commit changes, push them and demonstrate the difference between branches

Step 1: Add the new file.

```
git add program2.py
```

Step 2: Commit the changes.

```
git commit -m "Added greatest of two numbers program"
```

Step 3: Push the new branch to GitHub.

```
git push -u origin GIT-EXAM
```

Step 4: Check all branches.

```
git branch
```

Output:

```
* GIT-EXAM
  master
```

Step 5: Switch to master branch.

```
git checkout master
```

Step 6: Check files in master branch.

```
ls
```

Output:

```
program1.py
```

Step 7: Switch to GIT-EXAM branch.

```
git checkout GIT-EXAM
```

Step 8: Check files in GIT-EXAM branch.

```
ls
```

Output:

```
program1.py
program2.py
```

Step 9: Demonstrate differences between branches.

```
git diff master GIT-EXAM
```

This command displays the differences between the `master` and `GIT-EXAM` branches.

You can also compare commit history:

```
git log --oneline --all --graph
```

Important Git commands for viva

Command	Purpose
`git init`	Initialize repository
`git status`	Check repository status
`git add`	Add files to staging area
`git commit`	Save changes
`git push`	Upload changes to GitHub
`git pull`	Download latest changes
`git branch`	List branches
`git checkout`	Switch branches
`git checkout -b`	Create and switch branch
`git diff`	Compare changes
`git log`	View commit history
`git remote -v`	View remote repository URL

Important Viva Questions – Git and GitHub

Q1. What is Git?

Git is a distributed version control system used to track changes in source code.

Q2. What is GitHub?

GitHub is an online platform used to host Git repositories and collaborate on projects.

Q3. What is the difference between Git and GitHub?

- Git is a version control tool.

- GitHub is an online platform for hosting Git repositories.

Q4. What is the purpose of `git init`?

It initializes a new Git repository in the current project folder.

Q5. What is the difference between `git add` and `git commit`?

- `git add` moves changes to the staging area.
- `git commit` saves the staged changes in the repository.

Q6. What is the purpose of `git push`?

It uploads local commits to a remote repository like GitHub

Q7. What is a branch in Git?

A branch is a separate line of development that allows us to work on different features without affecting the main code.

Q8. What is the difference between `master` and `GIT-EXAM` branches in this assignment?

- `master` contains only `program1.py`.
- `GIT-EXAM` contains both `program1.py` and `program2.py`.

Q9. What is the purpose of `git diff master GIT-EXAM`?

It displays the differences between the `master` and `GIT-EXAM` branches.

Q10. What is the difference between `git checkout` and `git checkout -b`?

- `git checkout branch_name` switches to an existing branch.

- `git checkout -b branch_name` creates a new branch and switches to it.

Experiment 2: Git and GitHub Operations Using Git Bash

Aim: To perform Git operations using Git Bash, create a repository, push Python programs to GitHub, create a new branch, compare branches, and merge changes using a Pull Request.

a) Initialize a Git repository for a Data Science project

Step 1: Open Git Bash and create a project folder.

```
mkdir DataScienceProject
cd DataScienceProject
```

Step 2: Initialize Git repository.

```
git init
```

Step 3: Configure Git username and email.

```
git config --global user.name "Your Name"
git config --global user.email “your-email@gmail.com"
```

b) Push the repository to GitHub

1. Open GitHub.
2. Click on New Repository.
3. Enter repository name: `DataScienceProject`.
4. Select Public or Private.
5. Click Create Repository.

Connect your local repository to GitHub:

```
git remote add origin https://github.com/username/
DataScienceProject.git
```

Replace `username` with your GitHub username.

c) Create program1.py to check even or odd number

Step 1: Create a Python file.

```
touch program1.py
```

Step 2: Write the following Python program.

program1.py

```
num = int(input("Enter a number: “))

if num % 2 == 0:
     print(num, "is Even”)
else:
     print(num, "is Odd")
```

Output:

```
Enter a number: 10
10 is Even
```

d) Commit the code and push changes to GitHub

Step 1: Check repository status.

```
git status
```

Step 2: Add the file.

```
git add program1.py
```

Step 3: Commit the changes.

```
git commit -m "Added even odd program”
```

Step 4: Rename the default branch to master.

```
git branch -M master
```

Note: If your GitHub repository uses `main`, you can use `main` instead of `master`.

Step 5: Push changes to GitHub.

```
git push -u origin master
```

Step 6: Verify changes.

Open your GitHub repository and check whether `program1.py` is uploaded successfully.

e) Create a new branch GIT-EXAM and create program2

Step 1: Create and switch to a new branch.

```
git checkout -b GIT-EXAM
```

Step 2: Create a new Python file.

```
touch program2.py
```

Step 3: Write a Python program to find the reverse of a string.

program2.py

```
string = input("Enter a string: “)
reverse = string[::-1]
print("Reverse of string:", reverse)
```

Output:

```
Enter a string: hello
Reverse of string: olleh
```

f) Commit changes, push them and demonstrate differences between branches

Step 1: Add the new file.

```
git add program2.py
```

Step 2: Commit the changes.

```
git commit -m "Added string reverse program”
```

Step 3: Push the new branch to GitHub.

```
git push -u origin GIT-EXAM
```

Step 4: Check all branches.

```
git branch
```

Output:

```
* GIT-EXAM
  master
```

Step 5: Switch to master branch.

```
git checkout master
```

Step 6: Check files in master branch.

```
ls
```

Output:

```
program1.py
```

Step 7: Switch to GIT-EXAM branch.

```
git checkout GIT-EXAM
```

Step 8: Check files in GIT-EXAM branch.

```
ls
```

Output:

```
program1.py
program2.py
```

Step 9: Compare both branches.

```
git diff master GIT-EXAM
```

This command displays the differences between the two branches.

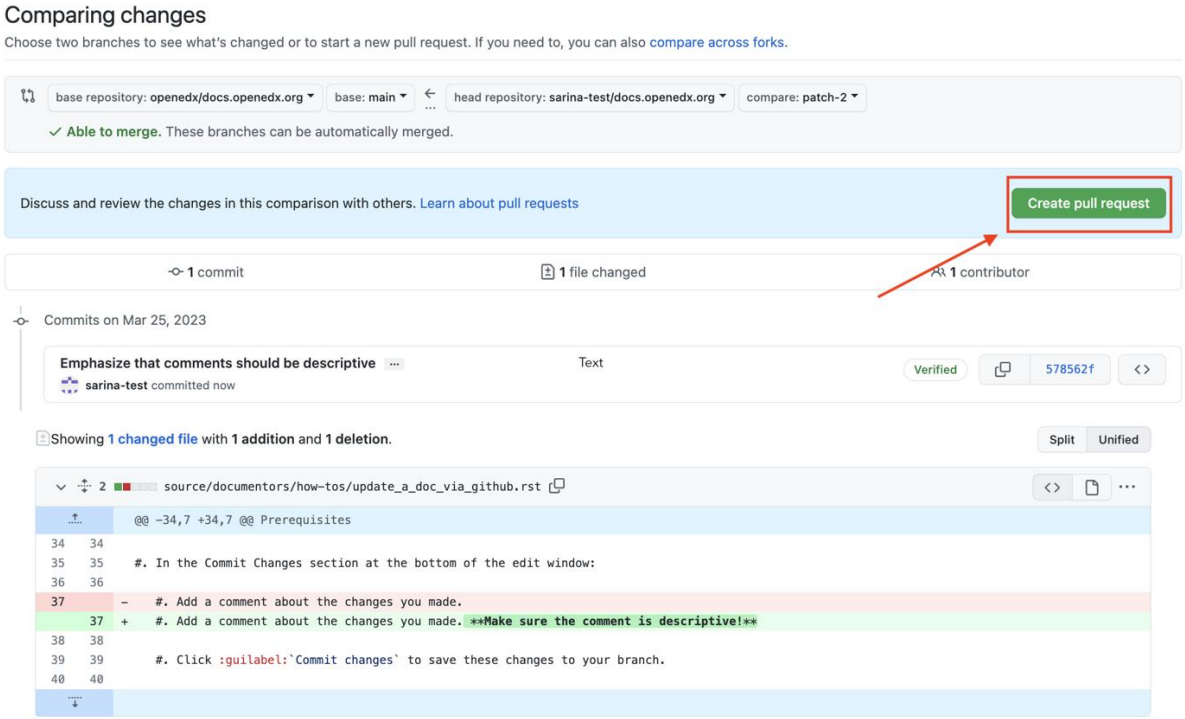
g) Create a Pull Request to merge GIT-EXAM into master/main

A Pull Request (PR) is a request to merge changes from one branch into another branch.

Step 1: Open your repository on GitHub.

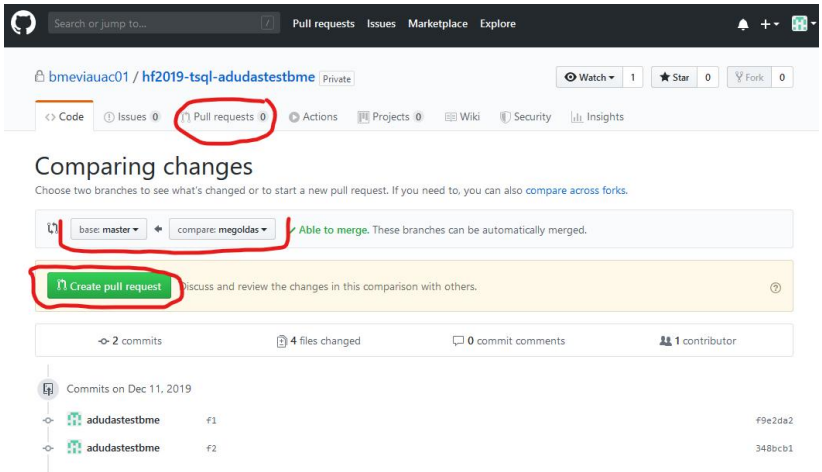
Step 2: Click on the Pull requests tab.

Step 3: Click on New pull request.



Step 4: Select the branches:

- Base: `master` (or `main`)
- Compare: `GIT-EXAM`



Step 5: Click on Create pull request.

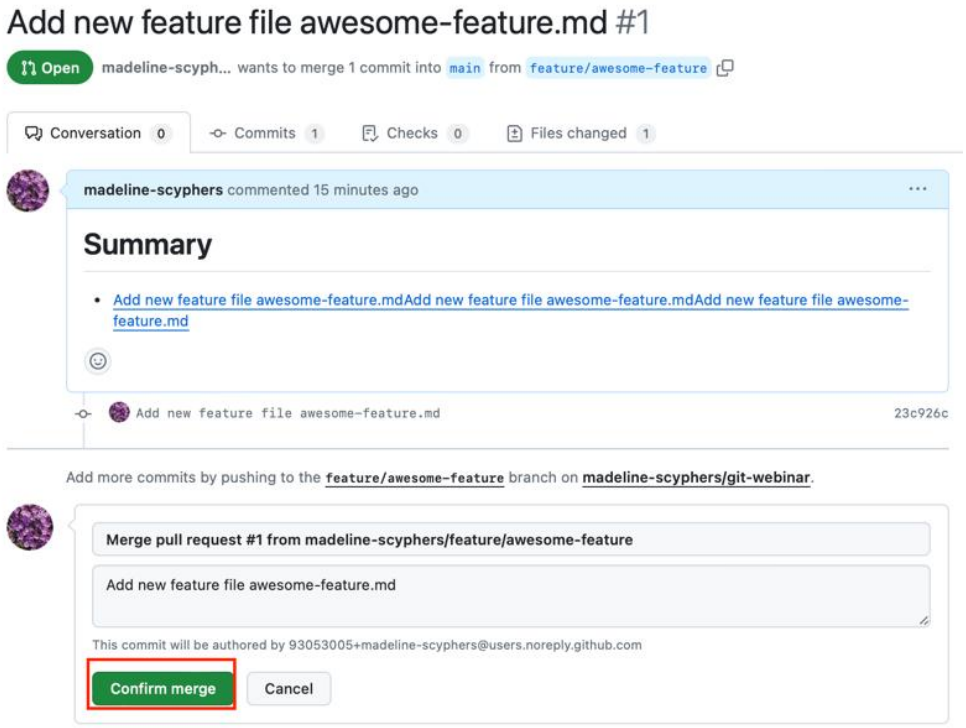
Step 6: Enter the title:

```
Merge GIT-EXAM into master
```

Step 7: Click on Create pull request.

Step 8: Click on Merge pull request.

Step 9: Click on Confirm merge.



Pull Request merged

Step 10: Verify the changes.

Open the GitHub repository and check that both `program1.py` and `program2.py` are now available in the master/main branch.

You can also update your local master branch:

```
git checkout master
git pull origin master
```

This downloads the latest changes from the `master` branch on GitHub and integrates them into your local `master` branch.

Important Git commands for viva

Command	Purpose
`git init`	Initialize repository
`git add .`	Add all changes
`git commit -m "message"`	Commit changes
`git push`	Push changes to GitHub
`git branch`	Display branches
`git checkout -b GIT-EXAM`	Create a new branch
`git diff master GIT-EXAM`	Compare branches
`git remote -v`	Display remote URL
`git pull`	Fetch and integrate remote changes

Q1. What is Git?

Git is a distributed version control system used to track changes in source code.

Q2. What is GitHub?

GitHub is an online platform used to store Git repositories and collaborate on projects.

Q3. What is the purpose of `git init`?

It initializes a new Git repository in the current project folder.

Q4. What is the difference between `git add` and `git commit`?

- `git add` moves changes to the staging area.
- `git commit` saves the staged changes in the local repository.

Q5. What is a branch in Git?

A branch is a separate line of development that allows us to work on different features without affecting the main code.

Q6. What is the purpose of `git checkout -b GIT-EXAM`?

It creates a new branch named `GIT-EXAM` and switches to it.

Q7. What is a Pull Request (PR)?

A Pull Request is a request to merge changes from one branch into another branch on GitHub.

Q8. What is the difference between `git push` and `git pull`?

- `git push` uploads local commits to GitHub.
- `git pull` fetches and integrates remote changes into the local repository.

Q9. What is the purpose of `git diff master GIT-EXAM`?

It displays the differences between the `master` and `GIT-EXAM` branches.

Q10. Why do we use `git pull origin master` after merging a Pull Request?

It updates the local master branch with the latest changes from GitHub so that both local and remote repositories remain synchronized.

Experiment 3: Git Branching, Pulling Changes, Merge Conflicts and Pull Request Using Git Bash

Aim: To perform Git operations using Git Bash, create multiple branches, push changes to GitHub, pull changes from another branch, demonstrate merge conflicts, compare branches, and merge changes using a Pull Request.

In this experiment, we will work with three branches:

1. `master` (or `main`)
2. `GIT-EXAM-1`
3. `GIT-EXAM-2`

a) Initialize a Git repository for a Data Science project

Step 1: Open Git Bash and create a project folder.

```
mkdir DataScienceProject
cd DataScienceProject
```

Step 2: Initialize Git.

```
git init
```

Step 3: Configure Git username and email.

```
git config --global user.name "Your Name"
git config --global user.email “your-email@gmail.com"
```

b) Push the repository to GitHub

1. Open GitHub.
2. Click on New Repository.
3. Enter repository name: `DataScienceProject`.
4. Click Create Repository.

Connect your local repository to GitHub:

```
git remote add origin https://github.com/username/
DataScienceProject.git
```

Replace `username` with your GitHub username.

Create an initial file so that the repository has a commit:

```
touch README.md
git add README.md

git commit -m "Initial commit”
```

Rename the default branch:

```
git branch -M master
```

Push it to GitHub:

```
git push -u origin master
```

c) Create GIT-EXAM-1 branch and write a palindrome program

Step 1: Create and switch to the new branch.

```
git checkout -b GIT-EXAM-1
```

Step 2: Create a Python file.

```
touch program2.py
```

Step 3: Write the palindrome program.

`program2.py`

```
string = input("Enter a string: “)
if string == string[::-1]:
    print("String is Palindrome”)
else:
    print("String is not Palindrome")
```

Output:

```
Enter a string: madam
String is Palindrome
```

What is a palindrome?

A palindrome is a word or string that reads the same forward and backward.

Examples: `madam`, `level`, `radar`.

d) Create program1.py to print squares of numbers from 1 to 10

Step 1: Create a Python file.

```
touch program1.py
```

Step 2: Write the following program.

```
program1.py
```

```
for i in range(1, 11):
    print("Square of", i, "=", i * i)
```

Output:

```
Square of 1 = 1
Square of 2 = 4
Square of 3 = 9
Square of 4 = 16
Square of 5 = 25
Square of 6 = 36
Square of 7 = 49
Square of 8 = 64
Square of 9 = 81
Square of 10 = 100
```

e) Commit the code and push changes to GitHub

Step 1: Check repository status.

```
git status
```

Step 2: Add both Python files.

```
git add .
```

Step 3: Commit the changes.

```
git commit -m "Added palindrome and square programs”
```

Step 4: Push GIT-EXAM-1 to GitHub.

```
git push -u origin GIT-EXAM-1
```

Step 5: Verify the changes.

Open your GitHub repository and select the `GIT-EXAM-1` branch.

You should see:

```
DataScienceProject/
    README.md
    program1.py
    program2.py
```

f) Create a new branch named GIT-EXAM-2

First, switch back to the master branch:

```
git checkout master
```

Create and switch to the second branch:

```
git checkout -b GIT-EXAM-2
```

Now you have two independent branches:

- `GIT-EXAM-1`
- `GIT-EXAM-2`

g) Pull changes from GIT-EXAM-1

Before pulling, we will make a change in `program1.py` on `GIT-EXAM-2` so that we can demonstrate a merge conflict.

Step 1: Create program1.py in GIT-EXAM-2.

```
touch program1.py
```

Step 2: Write a different program in program1.py.

For example, write a program to print cubes instead of squares:

```
for i in range(1, 11):
    print("Cube of", i, "=", i * i * i)
```

Step 3: Commit this change.

```
git add program1.py
```

```
git commit -m "Modified program1 in GIT-EXAM-2”
```

Step 4: Pull changes from GIT-EXAM-1.

```
git pull origin GIT-EXAM-1
```

Merge conflict may occur
Git will detect a conflict in `program1.py` because the same file was independently created with different contents in both branches.

h) Create program2.py in GIT-EXAM-2

When you pull changes from `GIT-EXAM-1`, the `program2.py` file is automatically brought into `GIT-EXAM-2`.

Important: You do not need to create it again because it already exists after pulling.

Check the files:

```
ls
```

Expected output:

```
README.md
program1.py
program2.py
```

The `program2.py` file contains the palindrome program:

```
string = input("Enter a string: “)
if string == string[::-1]:
    print("String is Palindrome”)
else:
    print("String is not Palindrome")
```

i) Modify program1.py to demonstrate merge conflicts

A merge conflict occurs when Git cannot automatically combine changes made to the same part of a file in different branches.

Step 1: Check the conflict status.

```
git status
```

You may see:

```
both added: program1.py
```

Step 2: Open program1.py.

You will see conflict markers similar to:

```
<<<<<<< HEAD
for i in range(1, 11):
    print("Cube of", i, "=", i * i * i)
=======
for i in range(1, 11):
    print("Square of", i, "=", i * i)
>>>>>>> GIT-EXAM-1
```

Explanation:

- `<<<<<<< HEAD` represents your current branch (`GIT-EXAM-2`).
- `=======` separates the two versions.
- `>>>>>>> GIT-EXAM-1` represents the incoming branch changes.

Step 3: Resolve the conflict.

Remove the conflict markers and write the final code.

For example:

```
for i in range(1, 11):
    print("Square of", i, "=", i * i)
```

Save the file.

Step 4: Add the resolved file.

```
git add program1.py
```

Step 5: Commit the resolved changes.

```
git commit -m "Resolved merge conflict in program1”
```

Now the conflict is resolved.

j) Commit changes, push them and demonstrate differences between branches

Step 1: Push GIT-EXAM-2 to GitHub.

```
git push -u origin GIT-EXAM-2
```

Step 2: Check all branches.

```
git branch
```

Expected output:

```
* GIT-EXAM-2
  GIT-EXAM-1
  master
```

Step 3: Compare master and GIT-EXAM-1.

```
git diff master GIT-EXAM-1
```

Step 4: Compare master and GIT-EXAM-2.

```
git diff master GIT-EXAM-2
```

Step 5: Compare GIT-EXAM-1 and GIT-EXAM-2.

```
git diff GIT-EXAM-1 GIT-EXAM-2
```

Step 6: Check commit history.

```
git log --oneline --all —graph
```

Difference between the three branches

Branch	Files available
master	README.md
GIT-EXAM-1	README.md, program1.py, program2.py
GIT-EXAM-2	README.md, program1.py, program2.py

Note: The final `program1.py` in `GIT-EXAM-2` contains the resolved version. `GIT-EXAM-1` retains its original version.

k) Create a Pull Request to merge GIT-EXAM-2 into GIT-EXAM-1

Before creating the Pull Request, make sure the resolved `program1.py` contains a change compared with `GIT-EXAM-1`. For example, use this final code so the Pull Request has a visible change:

```
for i in range(1, 11):
    print("Square of", i, "=", i * i)
    print("Cube of", i, "=", i * i * i)
```

Commit and push this final version:

```
git add program1.py
```

```
git commit -m "Updated program1 after conflict
resolution”
```

```
git push origin GIT-EXAM-2
```

Step 1: Open your GitHub repository.

Step 2: Click on the Pull requests tab.

Step 3: Click on New pull request.

Step 4: Select the branches:

Base branch (destination)

GIT-EXAM-1

Compare branch (source)

GIT-EXAM-2

Step 5: Click Create pull request.

Step 6: Enter the title:

```
Merge GIT-EXAM-2 into GIT-EXAM-1
```

Description:

```
Added palindrome program and modified program1.py after
resolving merge conflicts.
```

Step 7: Click Create pull request.

Step 8: Click Merge pull request.

Step 9: Click Confirm merge.

Pull Request successfully merged
Now the changes from `GIT-EXAM-2` are merged into `GIT-EXAM-1`.

Experiment 4: Git and GitHub Operations Using Git Bash (.gitignore)

Aim: To initialize a Git repository, create branches, push Python programs to GitHub, and use `.gitignore` to prevent selected files and directories from being uploaded to GitHub.

a) Initialize a Git repository

```
mkdir DataScienceProject
cd DataScienceProject

git init
```

Did: Created a project folder and initialized Git.

b) Connect repository to GitHub

1. Open GitHub.
2. Click New Repository.
3. Enter repository name: `DataScienceProject`.
4. Click Create Repository.

Connect local repository:

```
git remote add origin https://github.com/username/
DataScienceProject.git
```

Configure branch:

```
git branch -M master
```

Create initial commit:

```
touch README.md
git add .
git commit -m "Initial commit"
git push -u origin master
```

Did: Connected local repository to GitHub and pushed initial commit.

c) Create GIT-EXAM-1 branch and palindrome program

```
git checkout -b GIT-EXAM-1
```

Create file:

```
touch program2.py
```

Write the program:

program2.py

```
string = input("Enter a string: “)
if string == string[::-1]:
    print("String is Palindrome”)
else:
    print("String is not Palindrome")
```

Did: Created a branch and wrote a palindrome checking program.

d) Create program1.py to print squares from 1 to 10

```
touch program1.py
```

Write:

program1.py

```
for i in range(1, 11):
    print("Square of", i, "=", i * i)
```

Did: Created a program to print squares of numbers from 1 to 10.

e) Commit and push changes to GitHub

```
git status
git add .
git commit -m "Added palindrome and square programs"
git push -u origin GIT-EXAM-1
```

Did: Committed both Python files and pushed them to GitHub.

f) Create IGNORE directory and add text files

Create directory:

```
mkdir IGNORE
```

Create two text files:

```
touch IGNORE/test1.txt
touch IGNORE/test2.txt
```

Did: Created an IGNORE directory containing two text files.

g) Create .gitignore file and add IGNORE directory

Create `.gitignore`:

```
touch .gitignore
```

Open `.gitignore` and write:

```
IGNORE/
```

Did: Added the IGNORE directory to `.gitignore` so Git ignores its contents.

h) Commit, push and verify ignored files

Step 1: Check status.

```
git status
```

You should see `.gitignore` as an untracked file, but the `IGNORE` directory should not appear.

Step 2: Add and commit `.gitignore`.

```
git add .gitignore
git commit -m "Added gitignore file”
```

Step 3: Push changes to GitHub.

```
git push origin GIT-EXAM-1
```

Step 4: Verify on GitHub.

1. Open your GitHub repository.
2. Select the `GIT-EXAM-1` branch.
3. Check the files.

Expected repository structure:

```
DataScienceProject/
│
├── README.md
├── program1.py
├── program2.py
└── .gitignore
```

IGNORE directory excluded

The `IGNORE` directory and its `test1.txt` and `test2.txt` files will not appear on GitHub because they are ignored by Git.

Experiment 6: Forking, Branching and Pull Requests Using Git Bash

Aim: To clone an existing GitHub repository, demonstrate that direct pushing to the original repository is restricted, fork the repository, create a feature branch, modify code, push changes, and submit a Pull Request for review.

Repository: AI-Project-Gallery

Feature branch: `GIT-EXAM`

a) Clone the repository using Git Bash

Step 1: Open Git Bash.

Step 2: Navigate to Desktop.

```
cd Desktop
```

Step 3: Clone the original repository.

```
git clone https://github.com/KalyanM45/AI-Project-Gallery.git
```

Step 4: Enter the repository folder.

```
cd AI-Project-Gallery
```

Step 5: Check the remote repository.

```
git remote -v
```

Did: Downloaded a local copy of the original GitHub repository.

b) Demonstrate that branch creation is disallowed for this repository

Step 1: Create a new branch.

```
git checkout -b GIT-EXAM
```

Step 2: Try pushing the branch to the original repository.

```
git push -u origin GIT-EXAM
```

Expected result: GitHub rejects the push because you do not have write permission.

Example:

```
remote: Permission denied
fatal: unable to access repository
```

Did: Demonstrated that direct pushing to the original repository is restricted.

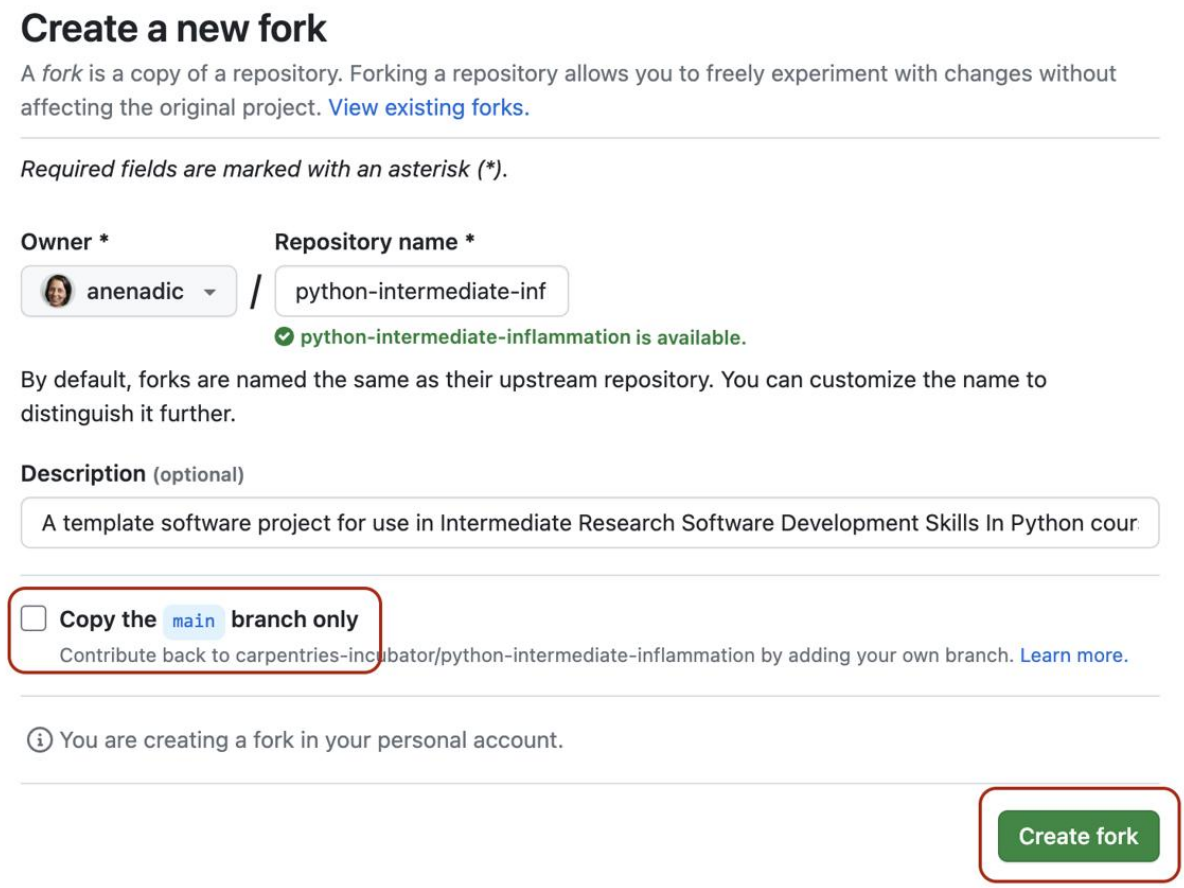
Important: Git allows you to create a local branch. What is restricted here is pushing that branch to the original repository without write permission.

c) Fork the AI project repository

A Fork creates your own copy of another user's repository under your GitHub account.

Steps:

1. Open the original repository on GitHub.
2. Click the Fork button.
3. Select your GitHub account
4. Keep the repository name as `AI-Project-Gallery`.
5. Click Create fork.



carpentries-incubator.github.io
Your repositories will look like this:

Repository	Owner
Original	KalyanM45
Forked	mansibansode806

Did: Created a personal copy of the original repository.

d) Create a feature branch named GIT-EXAM

Since the local branch already exists, configure your remotes first.

Step 1: Rename the original repository remote as `upstream`.

```
git remote rename origin upstream
```

Step 2: Add your forked repository as `origin`.

```
git remote add origin https://github.com/mansibansode806/AI-Project-Gallery.git
```

Step 3: Check remote URLs.

```
git remote -v
```

Expected:

```
origin    https://github.com/mansibansode806/AI-Project-Gallery.git\

upstream  https://github.com/KalyanM45/AI-Project-Gallery.git
```

Step 4: Check your current branch.

```
git branch
```

You should see:

```
* GIT-EXAM
  main
```

Did: Configured your fork as origin and the original repository as upstream.

e) Change or add your code in the branch

Step 1: Open the project in VS Code.

```
code .
```

Step 2: Open `README.md`.

Step 3: Find the existing AI/ML projects table.

It contains columns similar to:

- S No
- Project Name
- Domain
- Repository

- End-to-End

Step 4: Add a new project entry at the end of the table.

For example:

```
| 34 | Student Marks Prediction | Regression | [GitHub Repo](YOUR_REPOSITORY_URL) | ✔ |
```

Replace `YOUR_REPOSITORY_URL` with your actual project repository URL. Use the next available serial number in the existing table.

Step 5: Save the file using `Command + S`.

Step 6: Check modified files.

```
git status
```

Step 7: View the changes.

```
git diff
```

Did: Modified README.md by adding a new AI/ML project entry.

f) Submit a Pull Request for review

Step 1: Add the modified file.

```
git add README.md
```

Step 2: Commit the changes.

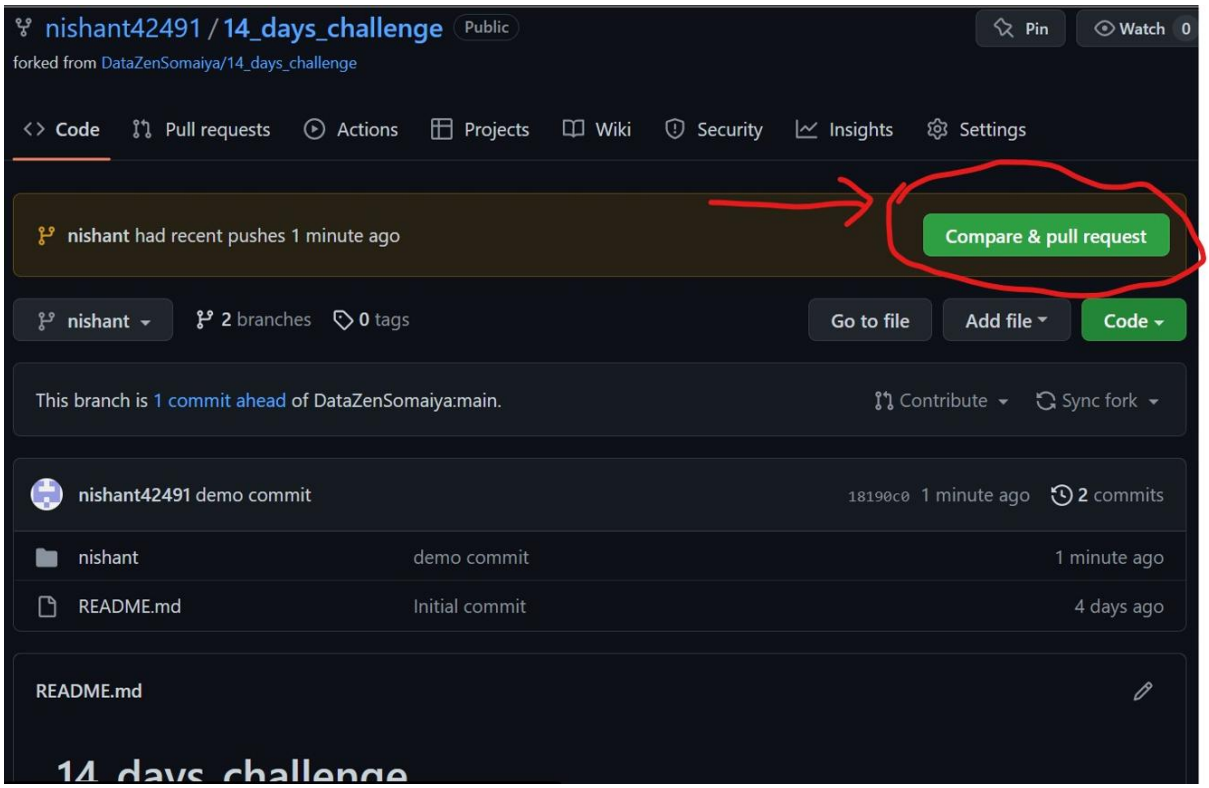
```
git commit -m "Added new AI project”
```

Step 3: Push the feature branch to your fork.

```
git push -u origin GIT-EXAM
```

Step 4: Open your forked repository on GitHub.

Step 5: Click Compare & pull request.



Step 6: Select the following branches:

Base repository

`KalyanM45/AI-Project-Gallery`

Base branch

`main`

Head repository

`mansibansode806/AI-Project-Gallery`

Compare branch

GIT-EXAM

Step 7: Enter the Pull Request details.

Title:

`Added new AI project`

Description:

```
Added a new AI/ML project entry to README.md.
```

Step 8: Click Create pull request.

Did: Submitted changes from your fork to the original repository for review.

In your actual attempt, GitHub displayed a message that the original repository owner had disabled Pull Request creation. Therefore, this step was blocked by the owner's settings.

g) Resolve merge conflicts and incorporate feedback

If the original repository allows Pull Requests, the maintainer may request changes or GitHub may report merge conflicts.

Step 1: Fetch the latest changes from the original repository.

```
git fetch upstream
```

Step 2: Switch to your feature branch.

```
git checkout GIT-EXAM
```

Step 3: Merge the latest original main branch.

```
git merge upstream/main
```

Step 4: If a merge conflict occurs, open the conflicting file in VS Code.

You may see:

```
<<<<<<< HEAD
Your changes
=======
Incoming changes
>>>>>>> upstream/main
```

Step 5: Remove the conflict markers and keep the required changes.

Step 6: Save the file and add the resolved changes.

```
git add .
```

Step 7: Commit the resolved changes.

```
git commit -m "Resolved merge conflicts”
```

Step 8: Push the updated branch.

```
git push origin GIT-EXAM
```

Step 9: If the reviewer requests modifications, make the changes and push again.

```
git add .
git commit -m "Addressed review feedback"
git push origin GIT-EXAM
```

Did: Resolved merge conflicts, incorporated review feedback, and updated the Pull Request.

Note: The original repository maintainer must approve and merge the Pull Request.

Important viva questions

Q1. What is Git?

Git is a version control system used to track changes in source code.

Q2. What is cloning?

Cloning means creating a local copy of a remote repository.

Q3. What is forking?

Forking means creating your own copy of another user's GitHub repository.

Q4. What is the difference between fork and clone?

- Fork creates a copy on GitHub.
- Clone downloads a repository to your local computer.

Q5. What is origin?

Origin is the default name commonly given to your remote repository, which in this experiment is your fork.

Q6. What is upstream?

Upstream refers to the original repository from which we forked.

Q7. What is a Pull Request?

A Pull Request is a request to merge changes from one branch or fork into another repository.

Q8. What is a merge conflict?

A merge conflict occurs when Git cannot automatically combine changes made to the same part of a file.

Q9. Why do we fork a repository?

We fork a repository to make our own copy so that we can modify code without directly changing the original repository.

Q10. Why was our Pull Request not created?

Because the original repository owner had disabled the ability to open Pull Requests.

Experiment 9: Docker

Aim: To deploy an AI model application using Docker by creating a Docker image and running it inside a Docker container.

We'll perform the entire experiment from the beginning on your MacBook Air M4, using Terminal and VS Code.

We'll use a simple Flask application for demonstration.

Step 1: Install Docker Desktop

Docker Desktop

Required software

Download Docker Desktop

1. Download Docker Desktop for Mac with Apple Silicon.
2. Open the downloaded `.dmg` file.
3. Drag Docker into Applications.
4. Launch Docker Desktop.
5. Complete the initial setup.
6. Wait until Docker Engine is running.

If the Rosetta installation popup appears, you can select Disable Rosetta for this experiment.

Verify installation

Open Terminal:

```
docker –version
```

Then:

```
docker run hello-world
```

Expected output:

```
Hello from Docker!
```

Step 2: Create a project folder

Open Terminal and execute:

```
mkdir DockerProject
cd DockerProject
```

Create the required files:

```
touch app.py requirements.txt Dockerfile
```

Your project structure:

```
DockerProject/
│
├── app.py
├── requirements.txt
└── Dockerfile
```

Step 3: Create the AI application (app.py)

Open the `DockerProject` folder in VS Code.

You can use:

```
code .
```

If the `code` command is unavailable, open the folder manually in VS Code.

Open `app.py` and write:

```
from flask import Flask, jsonify
```

```
app = Flask(__name__)

@app.route('/')
def home():
    return "AI Model Application is Running!"

@app.route('/predict')
def predict():
    prediction = "Positive"
    return jsonify({"Prediction": prediction})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Save the file.

Explanation:

- `Flask`: Creates a web application.
- `@app.route('/')`: Defines the home page.
- `@app.route('/predict')`: Defines the prediction API.
- `jsonify()`: Returns prediction in JSON format.
- `app.run()`: Starts the application on port 5000.
- `0.0.0.0`: Allows external access to the application inside the container.

Note: This is a demonstration prediction, not a trained machine learning model.

Step 4: Create requirements.txt

Open `requirements.txt` and write:

```
flask
```

Save the file.

Explanation: It contains the Python libraries required by the application. Docker will install Flask automatically during image creation.

Step 5: Create Dockerfile

Open `Dockerfile` and write:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Save the file.

Explanation of Docker instructions:

Instruction	Purpose
FROM	Selects Python base image
WORKDIR	Sets working directory
COPY	Copies files into container
RUN	Installs Flask
EXPOSE	Specifies application port
CMD	Executes the Python application

Step 6: Build Docker image

Go back to Terminal.

Make sure you are inside the `DockerProject` folder.

Execute:

```
docker build -t ai-model .
```

Explanation:

- `docker build`: Creates a Docker image.
- `-t ai-model`: Assigns a name to the image.
- `.`: Uses the current directory.

Wait for the build to finish.

To verify:

```
docker images
```

You should see `ai-model` in the image list.

Step 7: Run Docker container

First, make sure no previous container with the same name exists:

```
docker rm -f ai-container
```

If Docker says it does not exist, that's fine.

Now run:

```
docker run -d -p 5001:5000 --name ai-container ai-model
```

Container started

Why are we using 5001 instead of 5000?

Previously, your MacBook showed that port 5000 was already in use.

Therefore:

- `5001` = Host port (MacBook).
- `5000` = Container port (Docker).

This connects your MacBook to the Flask application inside Docker.

Step 8: Verify Docker container

Execute:

```
docker ps
```

You should see something similar to:

Field	Expected value
Image	ai-model
Status	Up
Ports	5001 → 5000

Name	ai-container

If the status is `Up`, your container is running.

Step 9: Test the application

Open your browser.

Test 1: Home page

Home page

Expected output
http://localhost:5001

```
AI Model Application is Running!
```

Open home page
Test 2: Prediction API

Prediction API

Expected output
http://localhost:5001/predict

```
{
  "Prediction": "Positive"
}
```

Open prediction API

Step 10: Stop the Docker container

After successful testing, execute:

```
docker stop ai-container
```

To remove the container:

```
docker rm ai-container
```

Your Docker image will remain available.